From CS229 to LLM Research

Six Portfolio Projects · A Plain-Language Roadmap · A Getting-Started Guide

Prepared for anish · bridging classical ML to LLM internals

August 2026

Contents

This single document has everything you need to begin. **Part I** is the plain-language map — what each of the six projects really is, the math to learn first, and a step-by-step path from *“I’ve read the notes”* to *“I finished the project.”* **Part II** is the concrete setup to do **first, tomorrow**: scaffolding the shared `llmkit` toolkit so you can read any model’s internal activations and next-token probabilities. **Part III** is the technical specification — exact equations, datasets, model IDs, milestones, and research questions for each project. Suggested reading order: skim Part I, do Part II at the keyboard, then work Part III one project at a time.

Part I · Plain-Language Guide & Roadmap

A friendly companion to `CS229_to_LLM_Research_Projects.md`. That file is the technical spec (exact equations, datasets, model IDs). This file is the map: what each project really is in everyday words, the math to learn first, the tools, and a step-by-step path from "I've read the notes" to "I finished the project."

How to use this guide

Read this **first**, then open the spec for the precise details when you're ready to build. For each project you'll find five things:

1. **In plain language** — what the project is, explained like I'm telling a friend over coffee.
2. **Math to learn first** — the prerequisites, in order, with the CS229 chapter that teaches each. Split into *must-know* and *nice-to-have*.
3. **Tools** — the actual software you'll use.
4. **The build roadmap** — a start-to-finish path: *learn this → build this → you're now at this level → connect it to the next piece → finish.*
5. **What you'll learn** — stated twice: once in ML vocabulary (for your résumé/interviews) and once in plain language (so you know what actually changed in your head).

One deliberate choice: the roadmaps tell you *what* to do at each step and *how to know you succeeded* — but they do **not** hand you the finished code or the experimental answers. That's on purpose. The understanding you're after lives in the doing; if I write the solution, you read it and forget it. Concrete dataset and model IDs are in the spec's Appendix F when you need them. Here, I'm giving you the path, not the destination.

A mindset that helps: every project is the same loop — *understand the math → write the algorithm yourself → point it at a real language model → look hard at what happens → explain the gap between what the theory promised and what you saw.* Once that loop feels natural, you're doing research.

The big picture: how the six projects connect

Don't think of these as six separate assignments. Think of them as one skill tree where each project hands tools to the next. Here's the journey.

Phase 0 — Foundations (before you touch a single project) **Learn:** refresh linear algebra (vectors, matrices, dot products) and the single most important classical algorithm — logistic regression, to the point where you can derive its gradient by hand. Skim CS229 Ch. 1–2.

Build: the shared `llmkit` toolkit from the spec — two small functions that (a) pull a model's hidden activations and (b) pull its next-token probabilities.

You're now at this level: *"I can open any small language model and read out both what it's 'thinking' (activations) and what it 'believes' (probabilities)."* This is the single unlock that makes every project possible.

Phase 1 — Project 1 (Probes): learn to *measure* a model You add a from-scratch logistic-regression classifier and use it as a measuring stick for what the model knows at each layer. **Connect:** the `probe.py` you write here gets reused in Projects 3 and 4 — so building it well now pays off twice later.

Phase 2 — Project 2 (Geometry): learn to *see* a model's representations You add linear-algebra tools (PCA, ICA, clustering) and study the *shape* of the model's embedding space. This is independent of Project 1 but shares the activation harness. **Connect:** dimensionality reduction here becomes a reflex you'll use to visualize anything high-dimensional in later work.

Phase 3 — Project 3 (Calibration): learn to *judge* a model Reusing your probe and the probability extractor, you ask whether the model's confidence is trustworthy — and fix it. **Connect:** it's the shortest project and reuses the most, so it's the perfect confidence-builder after the first two.

Phase 4 — Project 4 (Detection): learn to *compare* approaches You build several classifiers (generative and discriminative) on a real problem — spotting AI-written text — and watch the classic CS229 tradeoff play out live. **Connect:** this is where the whole first half of your notes comes together in one experiment.

Phase 5 — Project 5 (Double Descent & Grokking): learn how models *actually generalize* A change of pace — you train a tiny model from scratch and reproduce two famous, weird phenomena. **Connect:** this gives you the intuition for *why* big models behave the way they do, which reframes everything in Phases 1–4.

Phase 6 — Project 6 (RLHF): learn to *steer* a model The capstone. You implement the algorithm behind modern alignment and use it to change how a model writes. **Connect:** it combines generation, fine-tuning, and probability math from every earlier phase into the technique that defines today's frontier models.

The through-line: *measure → see → judge → compare → understand → steer.* Finish all six and you can honestly say you can take a language model apart and put it back together. Do even three (say P1, P3, P6 — one per skill) and you have a coherent story to tell.

Project 1 — Linear Probes: *measuring what a model knows*

In plain language A language model turns every word into a long list of numbers (a vector) at each of its internal layers. The question is: **is the model's knowledge actually sitting in those numbers, and where?** You test this by training a very simple classifier — the simplest one that exists — to read one specific thing (say, "is this sentence positive or negative?") off those numbers. If your simple classifier can do it, the model must be storing that information in a clean, easy-to-read way at that layer. By repeating this layer by layer, you draw a map of *where in the model different kinds of understanding appear*. It's like putting a stethoscope on different parts of the model and asking "can I hear sentiment here? how about grammar?"

Math to learn first *Must-know (learn these before starting):*

- **Vectors and matrices** — dot products, matrix–vector multiply. Any linear-algebra refresher.
- **The logistic (sigmoid) function and logistic regression** — CS229 Ch. 2.1. This *is* your probe.
- **Gradients / partial derivatives** — enough to understand "walk downhill to reduce error." CS229 Ch. 2.
- **The idea of maximum likelihood** — why we fit models by making the data probable. CS229 Ch. 1.3, 2.1.

Nice-to-have (you can pick these up as you go):

- **Newton's method** (using the second derivative to converge faster) — CS229 Ch. 2.4.
- **Regularization and cross-validation** (stopping a model from cheating) — CS229 Ch. 9.1, 9.3.
- **Bias-variance intuition** (why a *too-powerful* probe gives misleading answers) — CS229 Ch. 8.1.

Tools Python, PyTorch, Hugging Face `transformers` (to load the model), your own `llmkit` (activation extractor), NumPy (to write the classifier), matplotlib (for the layer map). No `scikit-learn` for the probe itself — you write that.

The build roadmap

1. **Learn:** derive the logistic-regression update rule until you can do it on paper. *(Checkpoint: you can explain why the gradient has the form "error × input.")*
2. **Build:** implement logistic regression from scratch in NumPy and test it on any toy 2-D data you make up. *(Checkpoint: it separates two blobs you can plot.)*
3. **Connect it to the model:** use `llmkit` to grab hidden vectors from one layer of a small model for a batch of sentences; feed those vectors as the input to your probe. *(Checkpoint: your probe beats "always guess the majority class" on held-out sentences.)*
4. **Level up — the honesty check:** re-run the probe against *shuffled/random* labels. If it still scores well, your probe is memorizing, not measuring. Report

your real score *minus* this control score. *(Checkpoint: you understand why raw accuracy alone would have fooled you.)*

5. **Complete:** sweep every layer, plot accuracy-vs-layer, and write two paragraphs on where the concept "lives" and whether that surprised you. **You're done when you have one clean figure and can defend it.**
6. **Carry forward:** keep your probe code tidy — Projects 3 and 4 import it.

What you'll learn *In ML terms:* linear classifiers, MLE and gradient-based optimization, the linear-probing methodology for interpretability, control tasks / probe selectivity, and layer-wise representation analysis.

In plain language: how to build the most fundamental classifier by hand, and how to use it as a tool to peek inside a model and prove *where* it stores a piece of knowledge — while avoiding the trap of fooling yourself with a number that looks good but means nothing.

Project 2 — The Shape of Meaning: *seeing a model's representations*

In plain language Every word and sentence lives as a point in a space with hundreds or thousands of dimensions. Humans can't picture that, so this project is about *compressing* that space down to a few directions we can actually look at and reason about — and asking whether those directions mean anything. You'll discover a strange fact: the model's "meaning space" isn't a nice round cloud; it's squished into a narrow cone, which quietly breaks the usual way people measure similarity. You'll fix that, and then go hunting for hidden "axes of meaning" (imagine finding that one direction in the space corresponds to "plural-ness" and another to "past tense"). It's detective work on the geometry of language.

Math to learn first *Must-know:*

- **Eigenvalues and eigenvectors, and covariance matrices** — the heart of PCA. CS229 Ch. 12.
- **Variance, and the difference between "uncorrelated" and "independent"** — this gap is the whole reason ICA exists. CS229 Ch. 13.1.
- **Clustering intuition** — grouping points by closeness. CS229 Ch. 10.

Nice-to-have:

- **SVD (singular value decomposition)** — another lens on PCA.
- **Probability densities and the change-of-variables formula** — the math under ICA. CS229 Ch. 13.2.
- **Latent variables, Jensen's inequality, and the EM idea** — for soft clustering with Gaussian mixtures. CS229 Ch. 11.

Tools Python, PyTorch + `transformers` (embeddings), `sentence-transformers` (for the similarity test), NumPy (you write PCA, ICA, k-means, and EM yourself), mat-

plotlib. `scikit-learn` allowed only as a *checker* — confirm your PCA matches theirs, then use yours.

The build roadmap

1. **Learn:** understand PCA as "find the directions of greatest variance," and be able to explain why those are the top eigenvectors of the covariance. *(Checkpoint: you can say what the eigenvalues mean.)*
2. **Build:** implement PCA two ways (full eigendecomposition, and power-iteration for just the top few). Sanity-check against a library to tolerance. *(Checkpoint: same answer as the library, your code.)*
3. **Connect it to the model:** run PCA on real embeddings; plot the variance spectrum and measure how "cone-shaped" (anisotropic) the space is. *(Checkpoint: you can state the anisotropy number.)*
4. **Level up — the payoff experiment:** remove the top few directions and remeasure how well cosine similarity matches human similarity judgments. Find the sweet spot. *(Checkpoint: you show similarity got* better* after deletion — a counterintuitive win.)*
5. **Go further:** run your ICA to look for interpretable independent axes, and cluster word senses with your EM/GMM. *(Checkpoint: you can name a handful of components that clearly mean something.)*
6. **Complete:** one figure contrasting "PCA directions (murky) vs. ICA directions (interpretable)," plus the similarity result. **Done when the figure tells the story without you narrating it.**

What you'll learn *In ML terms:* PCA/SVD, ICA and non-Gaussianity, k-means vs. Gaussian-mixture EM, whitening, and the anisotropy / representation-degeneration phenomenon in embeddings.

In plain language: how to take an incomprehensibly high-dimensional space and both *shrink it so you can see it* and *find the meaningful directions hidden inside it* — plus the humbling lesson that the "obvious" tool (PCA) isn't always the one that finds meaning.

Project 3 — Calibration: *judging whether a model is honest about doubt*

In plain language When a model says it's 90% sure, is it actually right 90% of the time? A trustworthy model's confidence should match its accuracy. Often it doesn't — especially after the "make it chatty and helpful" tuning step, which tends to make models overconfident (they sound sure even when they're guessing). This project measures that mismatch, draws the picture that reveals it, and then fixes it with a tiny, elegant adjustment — sometimes a *single number* — borrowed straight from your notes. It's about teaching yourself to tell the difference between a model that *knows* and a model that just *sounds* confident.

Math to learn first *Must-know:*

- **Probability distributions and expectation** — what a probability actually claims.
- **Softmax and cross-entropy** — how a model turns scores into probabilities and how it's trained. CS229 Ch. 2, Ch. 14 (Eq. 14.8).
- **Maximum likelihood again** — fitting by making observed outcomes probable.

Nice-to-have:

- **The exponential family / GLMs** — why softmax is the "natural" choice, not an arbitrary one. CS229 Ch. 3.
- **Newton's method in 1-D** — to fit the temperature parameter. CS229 Ch. 2.4.

Tools Python, PyTorch + `transformers` (to get answer-token probabilities via `llmkit`), NumPy (you write the temperature/Platt fitting and the error metric), matplotlib (reliability diagrams). `datasets` for the question banks.

The build roadmap

1. **Learn:** understand what "calibration" means and how it differs from accuracy — a model can be accurate but overconfident, or vice versa. *(Checkpoint: you can give an example of each.)*
2. **Build:** write the code that turns a batch of predictions + confidences into a **reliability diagram** and a single calibration-error number. *(Checkpoint: a perfectly-calibrated toy input gives near-zero error.)*
3. **Connect it to the model:** on a multiple-choice question set, read the probabilities the model assigns to each option and measure its real calibration. *(Checkpoint: you have the model's reliability diagram.)*
4. **Level up — the fix:** implement temperature scaling (fit one number by maximum likelihood on a validation split, using Newton's method) and show the error drop on a separate test split *without* hurting accuracy. *(Checkpoint: same accuracy, lower calibration error.)*
5. **The headline experiment:** compare a base model against its instruction-tuned twin. See whether tuning made it overconfident. *(Checkpoint: two reliability diagrams side by side.)*
6. **Complete:** write up "is this model honest about what it doesn't know, and what did tuning cost?" **Done when someone non-technical could read your diagram and get it.**

What you'll learn *In ML terms:* probability calibration, expected calibration error, reliability diagrams, temperature and Platt scaling, the softmax-as-GLM view, and the calibration cost of alignment tuning.

In plain language: how to check whether a model's confidence can be trusted, how to repair it with a shockingly small tweak, and the real-world insight that making a model friendlier can quietly make it more of a bluffer.

Project 4 — Who Wrote This?: *comparing generative and discriminative detectors*

In plain language Given a piece of text, can you tell if a human or an AI wrote it? This is a real, hard, important problem — and it's the perfect stage for the most important idea in the first half of your notes: there are **two philosophies of building a classifier.** One (generative) learns what human text and AI text each *look like* and asks "which does this resemble more?" The other (discriminative) skips the description and just learns the *dividing line* between them. Your notes claim each philosophy wins under different conditions — the descriptive one when you have little data, the line-drawing one when you have lots. You'll put that claim on trial. The clever twist: the best clues aren't the words themselves, but how *surprised the model is* by each word — AI text tends to be "too predictable." So you measure the model's surprise and classify on that.

Math to learn first *Must-know:*

- **Bayes' rule and conditional probability** — the engine of generative classifiers. CS229 Ch. 4.
- **The Gaussian and multivariate Gaussian** — how GDA models each class. CS229 Ch. 4.1.1.
- **Counting-based text models (multinomial) and smoothing** — the Naive Bayes baseline. CS229 Ch. 4.2.
- **The generative-vs-discriminative distinction** — the whole point. CS229 Ch. 4.1.3.

Nice-to-have:

- **Convex optimization + Lagrange duality** — to understand the SVM. CS229 Ch. 6.5.
- **The kernel trick** — bending a straight boundary into a curve. CS229 Ch. 5.
- **Coordinate ascent / the SMO algorithm** — how to actually train the SVM. CS229 Ch. 6.8.

Tools Python, PyTorch + `transformers` (to score how "surprised" a model is, via `llmkit` log-probabilities), NumPy (you write Naive Bayes, GDA, logistic regression, and the SVM/SMO), matplotlib. `datasets` for text.

The build roadmap

1. **Learn:** be able to explain, in one sentence each, how a generative and a discriminative classifier decide — and why they differ. *(Checkpoint: you can state the tradeoff from Ch. 4.1.3 out loud.)*
2. **Build the easy baseline:** implement multinomial Naive Bayes with smoothing on word counts. *(Checkpoint: it's above chance at separating human vs. AI text.)*
3. **Build the features that matter:** use `llmkit` to turn each text into a few "surprise" numbers (how likely the model found each word). *(Checkpoint: human and AI texts have visibly different surprise profiles when you plot them.)*

4. **The core comparison:** put GDA (generative) and logistic regression (discriminative) on the *same* surprise features, and plot how each does as you give them more and more training data. *(Checkpoint: you can see which one leads early and which one wins late — the notes' claim, tested.)*
5. **Level up:** implement the SVM via SMO from scratch and add a curved (kernel) boundary; see if it beats the linear methods. *(Checkpoint: your hand-written SMO trains and classifies.)*
6. **Complete:** the headline figure — accuracy vs. amount-of-data, with the crossover — plus a note on how detection gets harder as the AI writes more randomly. **Done when your figure demonstrates the tradeoff you read about.**

What you'll learn *In ML terms:* generative (GDA, Naive Bayes) vs. discriminative (logistic regression, SVM) modeling, the data-efficiency crossover, kernels and the SMO optimizer, and likelihood-based features for machine-text detection.

In plain language: the two fundamental ways to teach a machine to sort things, when to reach for each, and how to build all of them yourself — applied to a genuinely useful, current problem, using the idea that AI writing is "suspiciously unsurprising."

Project 5 — Double Descent & Grokking: *how models really learn to generalize*

In plain language Everything you're first taught about machine learning says: make your model too big and it will "overfit" — memorize the training data and fail on new data. Modern deep learning cheerfully breaks that rule, and this project lets you watch it happen with your own eyes on a model tiny enough to train on your laptop. You'll see **double descent** (as the model grows, performance gets worse, then — surprisingly — better again) and **grokking** (a model sits there having merely memorized the answers, and then, long after you'd have given up, it *suddenly understands the rule*). These aren't curiosities; they're clues to why giant models work at all. You'll reproduce them and then find the knob (a form of "keep the model humble") that controls them.

Math to learn first *Must-know:*

- **Least-squares and ridge regression** — the from-scratch "theory anchor" version of the effect. CS229 Ch. 1, Ch. 9.1.
- **The bias-variance decomposition** — the classic explanation for over/underfitting, and the algebra to split error into its parts. CS229 Ch. 8.1, 8.1.1.
- **What "overfitting" and "interpolation" (fitting training data perfectly) mean.** CS229 Ch. 8.2.

Nice-to-have:

- **How a small neural network and backprop work** — enough to train one. CS229 Ch. 7.

- **Weight decay / implicit regularization** — the control knob. CS229 Ch. 9.1, 9.2.

Tools Python, NumPy (the from-scratch linear/bias-variance experiments), PyTorch (the tiny Transformer — layers are plumbing, but you write the training loop and logging), matplotlib. **No dataset to download** — you generate simple math problems in a few lines.

The build roadmap

1. **Learn:** be able to write down the bias-variance decomposition and say what each term means. *(Checkpoint: you can explain why variance is what blows up when you overfit.)*
2. **Build the theory anchor:** reproduce double descent with plain (random-feature) ridge regression in NumPy — sweep model size across the "just barely fits the data" threshold and watch the error curve dip, spike, and dip again. *(Checkpoint: you've drawn the double-descent curve from scratch.)*
3. **Measure it:** estimate the bias and variance separately by training many models on resampled data. *(Checkpoint: the variance spike lines up with the error spike.)*
4. **Connect it to a real model:** train a tiny Transformer on a simple arithmetic rule and log train vs. test accuracy for a *long* time. *(Checkpoint: you catch the "grokking" moment where test accuracy suddenly jumps.)*
5. **Level up — find the knob:** vary the "keep it humble" setting (weight decay) and show it moves the grokking point and tames the spike. *(Checkpoint: you can make grokking happen sooner or later on demand.)*
6. **Complete:** one narrative — "the same phenomenon in three forms: simple theory → measured bias/variance → a real tiny Transformer." **Done when the three views clearly tell one story.**

What you'll learn *In ML terms:* bias-variance tradeoff and its decomposition, the interpolation threshold, model-wise and sample-wise double descent, grokking, and regularization's effect on generalization.

In plain language: why the old "bigger = overfitting" rule is incomplete, what really happens as models grow and train, and the hands-on intuition for the mysterious way large models suddenly "get it" — the closest thing to watching understanding form.

Project 6 — RLHF from Scratch: *steering how a model behaves*

In plain language This is how real labs turn a raw text-predictor into a helpful assistant: you let the model generate, you *score* what it produced (good/bad), and you nudge it to do more of the good. The scoring-and-nudging algorithm is called policy gradient, and it comes straight from the reinforcement-learning chapters of your notes. You'll build a miniature version — steering a small model to, say, write more positively — and along the way you'll hit the two truths every alignment team

knows: the naive method is *maddeningly noisy* (and there's a classic trick to calm it), and if you only chase the reward, the model finds dumb ways to cheat (so you add a leash that keeps it close to its original self). It's the most "real" project of the six — a tiny but faithful version of what powers today's chatbots.

Math to learn first *Must-know:*

- **Probability and expectation, and the "log-derivative trick"** — the one identity the whole method rests on. CS229 Ch. 17.
- **Gradients and gradient ascent** — you're climbing toward higher reward.
- **What a reward, a policy, and an MDP are** — the reinforcement-learning frame. CS229 Ch. 15.1.

Nice-to-have:

- **Variance of an estimator, and why a "baseline" reduces it** — the trick that makes training bearable. CS229 Ch. 17.
- **KL divergence** — the "leash" that stops the model drifting into gibberish. (Connects to regularization, Ch. 9.)
- **How autoregressive generation works** — the model's sampling *is* the policy. CS229 Ch. 14 (Eqs. 14.9–14.16).

Tools Python, PyTorch + `transformers` (the model you'll steer), `peft` (LoRA — lets you fine-tune cheaply within 8 GB), a small off-the-shelf reward scorer (or a simple rule you write), your `llmkit` for token probabilities, matplotlib.

The build roadmap

1. **Learn:** derive the REINFORCE gradient — understand *why* "reward × gradient-of-log-probability" nudges the model correctly. *(Checkpoint: you can explain the log-derivative trick.)*
2. **Build the loop with a fake-simple reward:** reward something trivial (like longer outputs) so you can debug the mechanics without a reward model. *(Checkpoint: the average reward climbs over training.)*
3. **Calm the noise:** add a baseline and measure how much steadier training becomes. *(Checkpoint: you can show the variance dropping, before vs. after.)*
4. **Use a real reward:** swap in a sentiment scorer and steer the model to write more positively. *(Checkpoint: before/after samples read differently.)*
5. **Level up — add the leash:** add a KL penalty toward the original model. Show that *without* it the model reward-hacks into nonsense, and *with* it, quality holds. *(Checkpoint: you can plot the reward-vs-drift tradeoff.)*
6. **Complete:** "RLHF in a few hundred lines," with reward curves, the noise-reduction result, the leash tradeoff, and sample outputs. **Done when you can explain every term in your loss to someone else.**

What you'll learn *In ML terms:* the policy-gradient theorem and REINFORCE, score-function estimators and baselines for variance reduction, KL-regularized fine-tuning, LoRA, and the RLHF training loop that underlies modern alignment.

In plain language: how to change a model's behavior by rewarding what you like — the actual method behind ChatGPT-style assistants — plus first-hand experience of the two things that make it hard: the training is jittery, and models will "cheat" any reward you don't guard carefully.

What you'll learn — in ML terms and in normal language

Here's the whole curriculum's payoff, said both ways: the left column is what you'd write on a résumé or say in a research interview; the right column is what actually changed in your understanding.

In ML / research terms	In normal language
Implemented logistic & softmax regression, MLE, and gradient/Newton optimization from scratch	I can build the most basic "learning" algorithms by hand and know *why* they work, not just how to call them
Applied linear probing and control tasks to analyze representations layer-by-layer	I can look inside a model and prove *where* it stores a piece of knowledge — without fooling myself
Performed PCA/ICA/SVD and EM-based clustering on embedding spaces; diagnosed anisotropy	I can shrink a huge, invisible space down to something I can see, and find the meaningful directions hidden in it
Measured and corrected model calibration (temperature/Platt scaling); analyzed the exponential-family view of softmax	I can tell whether a model's confidence is trustworthy, and fix it — and I learned that "helpful" tuning can make a model a bluffer
Compared generative vs. discriminative classifiers (GDA, Naive Bayes, logistic, kernel SVM via SMO) and the data-efficiency crossover	I know the two fundamental ways to teach a machine to sort things, when each wins, and I can code all of them
Reproduced double descent and grokking; decomposed error into bias and variance; studied regularization	I understand, hands-on, the strange truth about how modern models generalize — and why "bigger overfits" is wrong
Built a KL-regularized REINFORCE (policy-gradient) fine-tuning loop with variance-reduction baselines and LoRA	I can steer a model's behavior with rewards — the real technique behind assistants — and I've felt why it's noisy and why models cheat
Engineered a reusable interpretability toolkit; ran controlled experiments with baselines and reproducible figures	I can set up a clean experiment, rule out the boring explanation, and produce a result someone else can trust and repeat

And the meta-skill, in one sentence: you'll have practiced the actual loop of research — *turn math into code, point it at a real model, measure honestly, and explain*

the gap between theory and reality — which is exactly the muscle LLM research runs on.

This guide is the map; `CS229_to_LLM_Research_Projects.md` is the terrain (exact equations, datasets, model IDs, setup). Start with Phase 0, build the toolkit, and take the projects in order — each one hands tools to the next.

Part II · Getting Started — Scaffolding the `llmkit` Toolkit

Do this before Project 1. It takes one focused sitting (2–4 hours) and it unlocks all six projects. The goal of Phase 0 is small but crucial: ***be able to open any small model and read out both its internal activations and its next-token probabilities.*** *Everything else builds on that.*

> **What I give you here vs. what you build:** the toolkit is *plumbing* — shared infrastructure you're meant to reuse, so I hand you working code for it. The *learning algorithms* (logistic regression, PCA, ICA, GDA, SMO, REINFORCE, …) are left as stubs on purpose — coding those yourself is the whole point of the projects.

Your first sitting, hour by hour

- **Hour 1 — Environment.** Create the virtual environment, install the stack, confirm your GPU is visible to PyTorch.
- **Hour 2 — Repo skeleton.** Create the `llmkit/` package with the file layout below. Paste in the two working plumbing functions; leave the algorithm files as stubs.
- **Hour 3 — Smoke test.** Run the smoke-test script. When it prints activation shapes and a token log-probability for GPT-2, Phase 0 is done.
- **(Optional) Hour 4 — Warm-up.** Re-derive the logistic-regression gradient on paper so you walk into Project 1 ready to implement `probe.py`.

Definition of done for Phase 0: the smoke test runs on your RTX 4070 without error and prints (a) a hidden-state tensor shape like `(4, 768)` for four sentences, and (b) a per-token log-probability list for one sentence. That's it — you're ready for Project 1.

Step 1 — Environment

```
# from your projects folder
python3 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate

# PyTorch with CUDA (match the CUDA build to your driver; cu121 is a safe
 ↪  default)
pip install torch --index-url https://download.pytorch.org/whl/cu121

# the working stack
pip install transformers datasets accelerate
pip install numpy scipy matplotlib scikit-learn   # scipy/sklearn are for
 ↪  sanity-checks only
pip install sentence-transformers                  # used from Project 2 on
pip install peft bitsandbytes                       # LoRA + 4-bit; needed
 ↪  by Project 6 / big models
```

```
# confirm the GPU is visible
python -c "import torch; print('CUDA:', torch.cuda.is_available(), '|',
 ↪  torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU
 ↪  only')"
```

You should see your RTX 4070 printed. If `CUDA: False`, fix the PyTorch/CUDA install before continuing — the whole curriculum leans on the GPU.

Step 2 — The `llmkit` package layout

Create this exact tree:

```
llmkit/
  __init__.py       # re-export the main helpers
  activations.py    # [GIVEN] pull hidden states via forward pass / hooks
  logits.py         # [GIVEN] next-token log-probs + sequence log-likelihood
  probe.py       # [STUB]  you build this in Project 1 (logistic/softmax regression)
  linalg.py          # [STUB]  you build this in Project 2 (PCA / ICA / k-
means / EM)
  data.py           # [STUB]  small dataset loaders you add per project
  viz.py               # [STUB]  reliability diagrams, layer heatmaps, 2-
D projections
smoke_test.py       # [GIVEN] proves the setup works
```

`llmkit/__init__.py`:

```
from .activations import get_activations
from .logits import token_logprobs, sequence_loglik
```

Step 3 — The two functions that unlock everything (paste these in)

`llmkit/activations.py` — read the model's internal representations. This is the object your notes call `φ_θ(x)` (Ch. 14.1).

```
import torch

@torch.inference_mode()
def get_activations(model, tokenizer, texts, layers=None, pooling="last",
 ↪  batch_size=16):
    """
    Run `texts` through `model` and return hidden states at the chosen
 ↪  layers.

    Returns: dict {layer_index: FloatTensor (num_texts, hidden_dim)} on CPU.
```

```
    pooling: "last" (last real token) or "mean" (mean over real tokens).
    """
    device = next(model.parameters()).device
    if tokenizer.pad_token is None:                # e.g. GPT-2 has no pad
     ↪  token
        tokenizer.pad_token = tokenizer.eos_token
    out = {}
    for start in range(0, len(texts), batch_size):
        batch = texts[start:start + batch_size]
        enc = tokenizer(batch, return_tensors="pt", padding=True,
↪   truncation=True).to(device)
        hs = model(**enc, output_hidden_states=True).hidden_states  #
↪   tuple: (n_layers+1) x (B,T,H)
        chosen = range(len(hs)) if layers is None else layers
        mask = enc["attention_mask"]                # (B, T)
        for L in chosen:
            h = hs[L]                               # (B, T, H)
            if pooling == "mean":
                m = mask.unsqueeze(-1).float()
                pooled = (h * m).sum(1) / m.sum(1).clamp(min=1)
            else:  # "last" real token per sequence
                last_idx = mask.sum(1) - 1          # (B,)
                pooled = h[torch.arange(h.size(0)), last_idx]
            out.setdefault(L, []).append(pooled.float().cpu())
    return {L: torch.cat(v, 0) for L, v in out.items()}
```

`llmkit/logits.py` — read the model's next-token probabilities. These are the summands of the cross-entropy loss in your notes (Ch. 14, Eq. 14.8).

```
import torch

@torch.inference_mode()
def token_logprobs(model, tokenizer, text):
    """Per-token log p(x_t | x_<t) for one string.
    Returns (target_token_ids [T-1], logprobs [T-1])."""
    device = next(model.parameters()).device
    ids = tokenizer(text, return_tensors="pt").input_ids.to(device)   # (1,
↪   T)
    logits = model(ids).logits                                        # (1,
↪   T, V)
    logprobs = torch.log_softmax(logits[:, :-1].float(), dim=-1)      #
↪   predict token t+1
    targets = ids[:, 1:]                                          # (1, T-1)
    tok_lp = logprobs.gather(-1, targets.unsqueeze(-1)).squeeze(-1)   # (1,
↪   T-1)
    return targets[0].cpu(), tok_lp[0].cpu()
```

```
@torch.inference_mode()
def sequence_loglik(model, tokenizer, text, normalize=True):
    """Total (or per-token average) log-likelihood the model assigns to
     ↪  `text`."""
    _, lp = token_logprobs(model, tokenizer, text)
    return (lp.mean() if normalize else lp.sum()).item()
```

Bonus — forward hooks (for finer-grained internals). `output_hidden_states=True` gives you the residual stream at each layer, which covers Projects 1–4. When you later want a *specific* internal signal (e.g. an MLP's output for interpretability), register a hook:

```
captured = {}
def _save(name):
    def hook(module, inp, out): captured[name] = out.detach()
    return hook
h = model.transformer.h[6].mlp.register_forward_hook(_save("mlp_layer6"))
 ↪  # GPT-2 naming
# ... run a forward pass ...  then read captured["mlp_layer6"];  h.remove()
 ↪  when done
```

Step 4 — Leave these as stubs (you fill them in during the projects)

Creating the stubs now means your imports work and you have a clear "to-do" waiting. Example — `llmkit/probe.py`:

```
import numpy as np

class LogisticRegression:
    """From-scratch logistic regression. YOU implement this in Project 1.
    Fill in: fit() with BOTH gradient descent and Newton's method, plus L2;
    predict_proba(); predict(). Do NOT use sklearn here — deriving and
 ↪  coding
    the updates by hand is the entire point.
    """
    def __init__(self, l2=0.0):
        self.l2, self.w, self.b = l2, None, None

    def fit(self, X, y):
        raise NotImplementedError("Project 1 · Step 2: implement the
         ↪  gradient & Newton updates")

    def predict_proba(self, X):
        raise NotImplementedError

    def predict(self, X):
```

```
        raise NotImplementedError
```

Do the same "stub with a docstring + `NotImplementedError`" for `linalg.py` (PCA/ICA/k-means/EM — Project 2), and leave `data.py` / `viz.py` mostly empty; you'll grow them per project. The point: the scaffold runs today, and each project has an obvious home.

Step 5 — Smoke test (this is your Phase 0 finish line)

`smoke_test.py`:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from llmkit import get_activations, token_logprobs

name = "gpt2"                                   # ~124M params, trivial on 8 GB
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(
    name, torch_dtype=torch.float16
).to("cuda" if torch.cuda.is_available() else "cpu").eval()

sentences = [
    "The movie was absolutely wonderful.",
    "What a boring, terrible experience.",
    "Paris is the capital of France.",
    "The cat sat on the mat.",
]

# (a) activations: expect a (4, 768) matrix per layer
acts = get_activations(model, tok, sentences, layers=[6, 12],
 ↪  pooling="last")
for L, mat in acts.items():
    print(f"layer {L:>2}: activations {tuple(mat.shape)}")

# (b) token log-probs: the model's per-word 'surprise'
ids, lp = token_logprobs(model, tok, "The capital of France is Paris.")
print("mean token log-prob:", round(lp.mean().item(), 3))
print("per-token log-probs:", [round(x, 2) for x in lp.tolist()])
```

Run it:

```
python smoke_test.py
```

Expected: two lines like `layer  6: activations (4, 768)` and a mean log-prob plus a list of numbers. If you see those, **Phase 0 is complete** — commit the repo and start Project 1 tomorrow.

How Phase 0 connects to the rest

Everything downstream is a thin layer on these two functions:

- **Project 1 (Probes)** feeds `get_activations(...)` into the `LogisticRegression` you implement in `probe.py`.
- **Project 2 (Geometry)** feeds `get_activations(...)` into the PCA/ICA/EM you implement in `linalg.py`.
- **Project 3 (Calibration)** reads answer-token probabilities via `token_logprobs(...)`.
- **Project 4 (Detection)** turns `token_logprobs(...)` into "surprise" features for your from-scratch classifiers.
- **Projects 5–6** reuse the plumbing patterns (batched forward passes, log-probs) even though they add training loops.

Build this once, build it cleanly, and you never write model-plumbing again for the rest of the curriculum.

---

8 GB GPU habits (worth setting from day one)

- Wrap all inference in `torch.inference_mode()` (the toolkit already does).
- Load ≤1.4B models in `torch_dtype=torch.float16`; load 3–7B models with `load_in_4bit=True` (needs `bitsandbytes`) for inference only.
- Move captured tensors to **CPU immediately** (the toolkit does this) — a big batch of hidden states, not the weights, is what usually blows the 8 GB budget.
- Keep batches modest (8–16 short sentences) and truncate long inputs.
- For any training (Projects 5–6): use LoRA, gradient checkpointing, batch size 1–4 with gradient accumulation, and short sequences.

Part III · The Technical Specification

A project curriculum that turns the classical machine-learning math in your CS229 notes (Ng & Ma, 2023) into hands-on investigations of how large language models actually work.

Why this document exists

You already have the theory in `main_notes.txt`. The gap between "I read the derivation of the SVM dual" and "I understand it" is closed by **building the algorithm and pointing it at something you care about.** Your target is LLM research, so every project below does the same three things:

1. **Re-derive and hand-implement a classical method** from a specific chapter of your notes (from scratch, NumPy/PyTorch — no `sklearn.fit()` for the core algorithm).
2. **Apply it to a real artifact of a language model** — its embeddings, its hidden activations, its output probabilities, or its training dynamics.
3. **Ask a research-flavored question** that the experiment can actually answer, and write up what the theory predicted vs. what really happened.

This mirrors the philosophy of your CS229→quant curriculum, retargeted at LLMs:

> **Mathematics → From-Scratch Implementation → Real LLM Data → Controlled Experiment → Theory vs. Reality → A New Question.**

None of these are "download dataset, call library, report accuracy" projects, and none are the clichés (no MNIST, no Iris, no vanilla house prices). Each one is a thing you could put on a GitHub profile or discuss in a research interview.

How to read each project

Every project follows the same template so you can scan and compare:

- **Hook** — the one-sentence pitch.
- **Theme** — which of your four interests it serves (Internals · Behavior/Eval · Training Dynamics · Representation).
- **CS229 anchors** — exact chapters/sections/equations from `main_notes.txt` you'll exercise.
- **The bridge** — why the classical concept and the LLM phenomenon are the *same idea*.
- **The math you'll own** — what you should be able to derive on a whiteboard afterward.
- **Build it from scratch** — what you implement yourself vs. what's allowed to be a library ("plumbing").
- **Models & data** — concrete, and chosen to fit your 8 GB GPU.
- **Milestones** — a staged path from "minimum viable" to "portfolio-grade".

- **Theory vs. reality** — the specific claim from the notes you'll put on trial.
- **The research question** — the part that makes it yours.
- **Scope & pitfalls** — how to keep it to weeks, not years.
- **Effort** — a rough calendar estimate for one developer.

Your hardware, and what it means

Your box (i9-14900HX, **RTX 4070 Mobile / 8 GB VRAM**, 32 GB DDR5) is more than enough for this curriculum *if* you pick models deliberately. The 8 GB VRAM ceiling is the only real constraint. Rules of thumb used throughout:

- **Inference / activation extraction:** anything up to ~1.4B params runs in fp16 comfortably; 2.7B–7B runs in 4-bit (`bitsandbytes`) for inference with modest context lengths.
- **LoRA / QLoRA fine-tuning:** comfortable up to ~1.5B in fp16 LoRA, or ~3B in 4-bit QLoRA with small batch + gradient checkpointing. Keep RLHF-style training to ≤410M for a smooth ride.
- **Training a Transformer from scratch:** only tiny models (10^4–10^6 params, e.g. the grokking setup). That is by design in Project 5 — you do *not* need to train GPT-2 from scratch.
- **The Pythia model suite is your friend.** It ships the *same* architecture at 14M / 70M / 160M / 410M / 1B / 1.4B / 2.8B / 6.9B / 12B with 154 intermediate training checkpoints each. That lets you study scale and training-time effects by *downloading* checkpoints instead of training them — perfect for an 8 GB card.

A **model menu** you'll see referenced (all Apache/MIT-ish, all HF-hosted):

Need	Good picks (8 GB-safe)
Tiny, analyzable, many sizes	`EleutherAI/pythia-{70m,160m,410m,1.4b}`
Classic, well-studied internals	`gpt2`, `gpt2-medium`
Small but strong, instruction-tuned pairs	`Qwen/Qwen2.5-0.5B{,-Instruct}`, `Qwen/Qwen2.5-1.5B{,-Instruct}`
4-bit "big model" comparison	`meta-llama/Llama-3.2-3B`, `microsoft/phi-2`
Sentence embeddings	`sentence-transformers/all-MiniLM-L6-v2`

Your **"little bit of API access"** is reserved for *optional* comparison points (e.g. an API model that returns token log-probs, or generating a batch of machine-text samples). No project *requires* paid API calls.

> **Every dataset ID, model ID, split size, license note, and a copy-paste `load_dataset` snippet lives in Appendix F. Each project's "Models & data" block is the short version; Appendix F is the build-ready version.**

Build this once: the shared toolkit (a mini "Project 0")

Before Project 1, spend an evening building a small reusable package — `llmkit/` — that every later project imports. This is itself portfolio-worthy engineering and it stops you from rewriting the same plumbing six times.

```
llmkit/
  activations.py  # register forward hooks; return {layer: hidden_state} for a batch of pro
  logits.py      # next-token logits, log-probs of a target token, sequence log-
likelihood
   data.py           # loaders + tokenization for the datasets you'll reuse
  probe.py       # (filled in during Project 1) from-scratch logistic/softmax regression
  linalg.py      # (filled in during Project 2) power-iteration PCA, whitening, FastICA
  viz.py        # reliability diagrams, layer-vs-metric heatmaps, 2D projections
```

Two functions do most of the work and are worth getting right:

- **`get_activations(model, prompts, layers)`** — uses PyTorch **forward hooks** on the residual stream / MLP outputs to capture hidden states `φ_L(x) ∈ ℝ^d` at chosen layers. This *is* the representation `φ_θ(x)` from your notes' foundation-models chapter (§14.1) — the object the "linear probe" (Eq. 14.1) and "fine-tuning" (Eq. 14.2) are defined on.
- **`token_logprobs(model, text)`** — returns per-position `log p_θ(x_t | x_{<t})`, i.e. the summands of the autoregressive cross-entropy loss in your notes (Eq. 14.8). Half the projects are secretly about these numbers.

Everything else (tokenizers, `datasets`, `matplotlib`, training loops' outer scaffolding) is plumbing you may use freely. The *learning algorithms* are what you implement by hand.

The projects at a glance

#	Project	Theme	Core CS229 chapters	Headline LLM question
1	**Linear Probes & the Logit Lens**	Internals	1, 2, 8, 9 (+14.1)	*Where in an LLM does a concept become linearly readable?*

#	Project	Theme	Core CS229 chapters	Headline LLM question
2	**The Shape of Meaning** (PCA/ICA/EM on embeddings)	Representation	10, 11, 12, 13	*Is embedding space a narrow cone, and do its independent axes mean anything?*
3	**Are LLMs Honest About Uncertainty?** (calibration)	Behavior/Eval	1.3, 2, 3 (+14 temperature)	*Are token probabilities calibrated, and does instruction-tuning break that?*
4	**Who Wrote This?** (generative vs. discrimina-tive machine-text detection)	Behavior/Eval	4, 5, 6	*Does the GDA-vs-logistic tradeoff from the notes hold on AI-text detection?*
5	**Double Descent & Grokking** (from-scratch tiny Transformer)	Training Dynamics	7, 8, 9	*Can I reproduce double descent and grokking, and does weight decay control them?*
6	**RLHF from Scratch** (REINFORCE on a small LM)	Training Dynamics / Behavior	15, 17 (+14)	*Can policy gradient steer a language model's decoding, and what does it cost?*

Together they touch **Chapters 1–15 and 17**. (Chapter 16, LQR/DDP/LQG, is intentionally omitted — see the appendix for why it's the one piece that doesn't bridge cleanly to LLMs.)

Project 1 — Linear Probes & the Logit Lens: *where does a concept live?*

Hook. Train tiny linear classifiers on the frozen hidden states of an LLM to find out, layer by layer, where and when the model "knows" things like sentiment, part-of-speech, factual truth, or whether a sentence is grammatical — and turn the classical linear-model chapters into a working interpretability tool.

Theme. Internals & interpretability (with a side of representation geometry).

CS229 anchors. - **Ch. 2 — Logistic & softmax regression** (§2.1 logistic regression, §2.3 multi-class/softmax, §2.4 Newton's method). Your probe *is* logistic/softmax regression. - **Ch. 1 — Linear regression** (§1.2 normal equations, §1.3 the probabilistic interpretation) for continuous probes (e.g. probing for token position or sentence length). - **Ch. 9 — Regularization & model selection** (§9.1 L2 regularization, §9.3 cross-validation) — essential, because a probe that's too powerful "reads in" structure that isn't there. - **Ch. 8 — Generalization / bias-variance** (§8.1) to reason about *selectivity*: a probe's accuracy confounds "the info is in the representation" with "the probe is strong." - **Ch. 14 — Foundation models** (§14.1, Eq. 14.1): the **linear probe** is literally defined there as `w^⊤ φ_θ(x)` on a frozen representation. You are implementing the notes' own equation.

The bridge. The notes define adaptation of a foundation model either by **fine-tuning** (Eq. 14.2, update everything) or by a **linear probe** (Eq. 14.1, a linear head on frozen features). Interpretability research uses that exact linear probe not to *solve* a task but to *ask a question*: if a simple linear map from layer L's activations predicts property P with high accuracy, then P is (linearly) represented at layer L. So logistic regression — Chapter 2 — becomes a measuring instrument for the geometry of thought inside a Transformer.

The math you'll own. Deriving the logistic-regression gradient and the Newton update (§2.4); the softmax cross-entropy gradient for the multi-class case (§2.3); why L2 regularization (§9.1) corresponds to a Gaussian prior / MAP estimate (ties to §9.4); the bias-variance reason a high-capacity probe overstates how much a layer "knows."

Build it from scratch. - Logistic regression via (a) batch gradient descent and (b) Newton–Raphson (§2.4) — implement both and compare convergence. This becomes `llmkit/probe.py`. - Softmax regression for multi-class probes (POS tagging). - L2 regularization + a from-scratch k-fold cross-validation loop (§9.3) to set the penalty. - A **selectivity control**: the "control task" trick — re-run the probe against *random* labels; report accuracy *minus* control accuracy, not raw accuracy. (This is the rigorous version and what separates a portfolio project from a toy.) - *Plumbing allowed:* HF model loading, tokenization, `get_activations` hooks, plotting.

Models & data. - Models: `gpt2` and `pythia-410m` (both fp16 on 8 GB, easy). Optionally repeat on `pythia-70m` vs `pythia-1.4b` to see how "where a concept lives" shifts with scale. - Probing targets (pick 3–4): binary **sentiment** (SST-2), **part-of-speech** (any Universal Dependencies treebank), **factual truth** of simple statements (the *cities*/*true-false* style datasets used in "geometry of truth" work — or build your own 500 true/false statements), and a **syntactic** property (subject–verb number agreement).

Milestones. 1. *MVP:* logistic-regression probe on the last layer of GPT-2 for sentiment; beats majority baseline; cross-validated penalty. 2. *Layer sweep:* probe every layer; produce the signature **layer-vs-accuracy curve**. Add the control-task selectivity correction. 3. *Logit lens:* project mid-layer activations through the model's own unembedding matrix to read the "running guess" of the next token; compare where the probe says info appears vs. where the logit lens says it's usable. 4. *Portfolio-grade:* repeat across 2–3 model sizes; write up "concept X becomes linearly decodable around relative depth d," with selectivity-corrected curves and honest error bars.

Theory vs. reality. The notes present the linear probe as a pragmatic adaptation method. You'll test the *interpretability* reading: does accuracy rise then plateau/fall across depth (the widely reported "concepts peak in the middle" pattern)? Does Newton's method (§2.4) actually converge in far fewer steps than GD here, as the theory promises, or does the high dimensionality (d≈768–2048) change the story?

The research question. *"For a given concept, does the layer of peak linear decodability move earlier or later as model scale grows — and is that shift the same for surface features (POS) vs. semantic features (truth)?"* A clean, novel-enough finding you can plot and defend.

Scope & pitfalls. Don't let the probe be an MLP — that defeats the measurement. Watch for **train/test leakage** through tokenization (probe the representation of a *fixed* token position). Balance your classes. The control-task correction is the single most important thing reviewers will look for.

Effort. ~2–3 weekends. (MVP in a day once `llmkit` exists.)

Project 2 — The Shape of Meaning: PCA, ICA & EM on embedding space

Hook. Take an LLM's embeddings and interrogate their geometry from scratch: implement PCA by power iteration, discover the notorious "anisotropy" (embeddings crammed into a narrow cone), test whether *deleting* the top principal components makes semantic similarity **better**, then use ICA to hunt for independent, interpretable semantic axes and EM/GMM to induce word senses.

Theme. Representation & data geometry (with interpretability payoff).

CS229 anchors. - **Ch. 12 — PCA** (the whole chapter): covariance, eigenvectors, the maximal-variance derivation. - **Ch. 13 — ICA** (§13.1 ambiguities, §13.2 densities under linear transforms, §13.3 the Bell–Sejnowski update rule). ICA finds statistically *independent* components, not just uncorrelated ones. - **Ch. 10 — k-means** and **Ch. 11 — EM for mixtures of Gaussians** (§11.1, §11.4; Jensen's inequality §11.2; the ELBO §11.3). Cluster embeddings to induce senses/topics with soft assignments.

The bridge. Your notes describe word/token **embeddings** `e_i ∈ ℝ^d` and contextual representations `φ_θ(x)` (§14.3, §14.1) as the numeric substrate of language models. Chapters 10–13 are precisely the toolkit for asking *what shape that substrate has*. Two real, somewhat surprising phenomena make this more than a demo: (1) **representation anisotropy** — contextual embeddings occupy a narrow cone, so raw

cosine similarity is misleading, and simply removing the top few PCA directions improves semantic tasks ("all-but-the-top"); (2) recent work shows **ICA** recovers embedding axes that are far more human-interpretable than PCA axes and even align across languages/models. You'll reproduce both from scratch.

The math you'll own. PCA as eigendecomposition of the covariance / as SVD, and why the top directions capture max variance (Ch. 12); why decorrelation (PCA) ≠ independence, and how ICA's non-Gaussianity objective (Ch. 13) gets you the rest; EM as coordinate ascent on the ELBO and why it monotonically increases the likelihood (§11.2–11.3).

Build it from scratch. - **PCA** two ways: (a) eigendecomposition of the covariance matrix, (b) **power iteration / deflation** for the top-k components (great for intuition and for high-d embeddings). Add whitening. - **ICA** via the update rule in §13.3 (gradient ascent on the log-likelihood with a sigmoid-CDF source model) — the notes literally hand you the algorithm. A FastICA variant is a fine stretch. - **k-means** and **GMM via EM** (full/diagonal covariance), with the ELBO tracked per iteration to *show* monotonic improvement (a satisfying plot that proves your EM is correct). - *Plumbing:* embedding extraction, `datasets`, 2-D layout for figures.

Models & data. - Static token embeddings from `gpt2` / `pythia` embedding matrices, **and** contextual embeddings from mid/late layers via `get_activations`. - Sentence embeddings from `all-MiniLM-L6-v2` for the semantic-similarity evaluation (STS-B pairs) so you can *quantify* whether "all-but-the-top" helps.

Milestones. 1. *MVP:* PCA (power iteration) on token embeddings; visualize the variance spectrum; measure anisotropy (mean cosine similarity of random pairs; the "cone" statistic). 2. *All-but-the-top:* remove top-k components, re-measure STS correlation; find the k that maximizes it. Report the win. 3. *ICA hunt:* run your ICA; inspect the top-activating tokens per independent component; label the ones that are interpretable ("plural nouns," "months," "code tokens"…). 4. *Senses via EM:* pick polysemous words ("bank," "spring," "cell"), cluster their contextual embeddings with your GMM; show separated senses and compare to k-means (soft vs. hard). 5. *Portfolio-grade:* one figure comparing PCA axes (uninterpretable) vs. ICA axes (interpretable) on the same data, plus the anisotropy-correction result with numbers.

Theory vs. reality. The notes motivate PCA as *the* dimensionality-reduction method; reality is that its top directions here are often dominated by frequency/anisotropy artifacts and are *not* the semantically interesting ones — ICA's independence criterion (Ch. 13) does better. You'll show, with your own code, *why* "uncorrelated" (PCA) is weaker than "independent" (ICA).

The research question. *"Do the independent components recovered by ICA on one model's embeddings correspond to the same semantic axes recovered on another model's embeddings?"* (A from-scratch, small-scale echo of the "universal geometry" result — a genuinely current research thread.)

Scope & pitfalls. ICA is sensitive to preprocessing — whiten first (PCA feeds ICA naturally). Don't over-interpret components; report the fraction that are human-labelable honestly. GMM in full dimension is unstable — reduce with your PCA first (a nice pipeline that reuses your own code).

Effort. ~3 weekends. This is the most "classical ML" of the six and the best showcase of Chapters 10–13.

Project 3 — Are LLMs Honest About Uncertainty? Calibration, temperature & the exponential family

Hook. Measure whether an LLM's stated probabilities mean what they say — when it assigns 0.8 to an answer, is it right 80% of the time? — then fix miscalibration from scratch with temperature and Platt scaling, and connect the whole story back to logistic regression and the exponential family.

Theme. Behavior & evaluation.

CS229 anchors. - **Ch. 3 — Generalized linear models** (§3.1 the exponential family, §3.2 constructing GLMs). The softmax over next-token logits is the canonical response function of the **categorical GLM**; understanding this is the theoretical spine of the project. - **Ch. 2 — Logistic regression** (§2.1) and **Newton's method** (§2.4) — Platt scaling *is* 1-D logistic regression on logits; temperature scaling is a 1-parameter fit you'll solve with Newton's method. - **Ch. 1 — Probabilistic interpretation** (§1.3): the notion that a model outputs a *distribution*, and MLE, is exactly the frame calibration lives in. - **Ch. 14 — Temperature** (Eqs. 14.13–14.16): your notes already introduce `softmax(f_θ(·)/τ)` and explain how τ sharpens/flattens the distribution. Calibration is the principled way to *choose* τ.

The bridge. The autoregressive LLM defines `p_θ(x_t | x_{<t}) = softmax(f_θ(·))` (Eq. 14.7) and is trained by cross-entropy MLE (Eq. 14.8) — pure Chapter 2/3 machinery at scale. But minimizing cross-entropy does **not** guarantee the probabilities are *calibrated*, especially after instruction-tuning/RLHF. So you take the exponential-family/GLM view of the model's head and ask an empirical question the notes set up but don't answer: *are these probabilities trustworthy, and can a 1-parameter GLM-style recalibration fix them?*

The math you'll own. Why softmax is the exponential family's categorical link (§3.1–3.2); the MLE/Newton derivation for fitting a temperature; the definition and estimator of **Expected Calibration Error (ECE)** and reliability diagrams; the distinction between *calibration* and *accuracy* (you can improve one without the other).

Build it from scratch. - **Reliability diagrams + ECE / adaptive-ECE** estimators. - **Temperature scaling**: fit a single τ by minimizing NLL on a held-out set via Newton's method (§2.4) — derive the 1-D gradient/Hessian yourself. - **Platt / vector scaling**: logistic / multinomial-logistic regression on the logits (reuse `llmkit/probe.py` from Project 1 — nice payoff). - A **selective-prediction** curve (accuracy vs. coverage when you abstain below a confidence threshold) — the practical reason calibration matters. - *Plumbing:* extracting answer-token logits/log-probs via `llmkit/logits.py`.

Models & data. - Multiple-choice QA where the answer is a single token (A/B/C/D): a subset of **MMLU**, **ARC**, or **HellaSwag**. Read off the log-probs of the option tokens. - The key comparison: a **base vs. instruction-tuned pair** at the same size — e.g. `Qwen2.5-1.5B` vs `Qwen2.5-1.5B-Instruct`, or `pythia` base vs a tuned variant. This

isolates *what tuning does to calibration*. - *Optional API touch:* if you have access to a model that returns log-probs, add it as a third point on the plot.

Milestones. 1. *MVP:* reliability diagram + ECE for one base model on 500 MMLU questions. 2. *Recalibrate:* fit temperature (Newton) and Platt scaling on a validation split; show ECE drop on test with accuracy unchanged. 3. *The instruction-tuning effect:* base vs. instruct reliability diagrams side by side. Is the tuned model **overconfident**? (Commonly yes.) 4. *Portfolio-grade:* selective-prediction curves, calibration across subjects/difficulty, and a short written claim about the accuracy-vs-honesty tradeoff of alignment tuning.

Theory vs. reality. MLE training (the notes' Eq. 14.8) is often assumed to yield meaningful probabilities. You'll show empirically where that breaks — and that a *single scalar* from a GLM recalibration (Chapter 3) recovers much of the honesty, which is a striking "theory earns its keep" moment.

The research question. *"Does alignment/instruction-tuning systematically trade calibration for helpfulness, and is the damage uniform across topics or concentrated where the model is being 'agreeable'?"*

Scope & pitfalls. Keep answers to single tokens (multi-token answers need length normalization — a rabbit hole). Separate the calibration-fit split from the test split. Report accuracy alongside ECE so you don't "improve" calibration by hurting accuracy.

Effort. ~2 weekends. High insight-per-hour; excellent interview talking point.

Project 4 — Who Wrote This? Generative vs. discriminative detectors of machine text

Hook. Build detectors that tell human text from LLM-generated text, and use the problem as a live arena for the single most important comparison in your notes — **generative (GDA / Naive Bayes) vs. discriminative (logistic regression / SVM)** — including the elegant idea of using an LLM's *own* log-probabilities as the features.

Theme. Behavior & evaluation (and a full showcase of the generative-learning chapter).

CS229 anchors. - **Ch. 4 — Generative learning** (§4.1 GDA, §4.1.1 the multivariate normal, §4.1.3 the GDA-vs-logistic-regression discussion, §4.2.2 the multinomial event model for text, §4.2.1 Laplace smoothing). This chapter is *literally about classifying text* — you're applying it to a 2024-era version of the task. - **Ch. 2 — Logistic regression** as the discriminative counterpart. - **Ch. 5 — Kernels** and **Ch. 6 — SVM** (§6.5 duality, §6.8 the SMO algorithm): a kernel SVM is the third detector, and implementing SMO from scratch is a rite of passage.

The bridge. Detecting machine-generated text is a real, unsolved, high-stakes problem. It's also the perfect testbed for §4.1.3's central claim: *generative models (GDA/NB) make stronger assumptions and win with little data; discriminative models (logistic/SVM) make weaker assumptions and win with lots.* The twist that makes it

LLM-flavored: the best features aren't raw words but the **per-token log-likelihood signature** from a language model (Eq. 14.8's summands) — human text and model text sit differently under a model's own distribution (the intuition behind DetectGPT-style curvature detectors). So you compute features *with* an LLM and classify them *with* Chapter 4/6 algorithms you wrote yourself.

The math you'll own. The GDA MLE (fitting φ, μ_0, μ_1, shared Σ) and why a shared covariance yields a *linear* boundary that connects to logistic regression (§4.1.3); the multinomial Naive Bayes event model and Laplace smoothing (§4.2); the SVM dual and the SMO coordinate-ascent updates (§6.8); the kernel trick (§5.3) for a nonlinear boundary in log-prob-feature space.

Build it from scratch. - **Naive Bayes** multinomial event model + Laplace smoothing (§4.2) on n-gram counts — the fast baseline. - **GDA** on continuous feature vectors (fit the Gaussians; derive the linear/quadratic boundary). - **Logistic regression** (reuse `llmkit/probe.py`). - **Kernel SVM via SMO** (§6.8) — the from-scratch centerpiece; RBF and linear kernels. - **The LLM feature extractor**: for each text, compute features from `token_logprobs` — mean log-prob, log-prob variance, "curvature" (drop in likelihood under small perturbations à la DetectGPT), rank statistics. This is where the LLM enters. - *Plumbing:* text datasets, tokenization, perturbation generation.

Models & data. - A "scorer" LM for features: `gpt2` / `pythia-410m` (you only need log-probs, so this is cheap). - Data: pair **human** text (e.g. WebText, Wikipedia, or human essays) with **machine** text you generate yourself from a small model at several **temperatures** (reuse Eqs. 14.13–14.16!) and, optionally, a few samples from an API model.

Milestones. 1. *MVP:* Naive Bayes on n-grams, human vs. machine; ROC/AUC baseline. 2. *Generative vs. discriminative:* GDA vs. logistic regression on the *same* log-prob features; plot AUC as you **vary training-set size** — this is the §4.1.3 claim on trial. 3. *SVM/SMO:* add your kernel SVM; compare linear vs. RBF on the log-prob features. 4. *Stress test:* how does every detector degrade as the *generator's* temperature rises, and across generator sizes? (Higher temperature → more human-like → harder.) 5. *Portfolio-grade:* one figure of AUC-vs-data-size showing the generative model leading in the low-data regime and the discriminative model overtaking it — your notes' theorem, empirically, on a modern problem.

Theory vs. reality. §4.1.3 predicts a **crossover**: NB/GDA better with few examples, logistic/SVM better asymptotically. Does it actually cross over on this task? Where? Does the crossover move when features are LLM log-probs vs. raw n-grams?

The research question. *"Are LLM-log-probability features 'Gaussian enough' for GDA's assumptions to pay off in the low-data regime, and does the generative-vs-discriminative crossover from CS229 survive when the classifier's features are themselves produced by a language model?"*

Scope & pitfalls. Detector performance is very sensitive to domain/topic leakage — match human and machine text on topic/length or you'll "detect" the topic, not the author. Keep the perturbation-curvature feature simple (a few masked-token resamples) to avoid a compute blowup. Report AUC, not just accuracy (classes/threshold matter).

Effort. ~3–4 weekends (SMO is the time sink, and worth it).

Project 5 — Double Descent & Grokking: bias-variance on a Transformer you can actually train

Hook. Reproduce two of the most counterintuitive phenomena in modern deep learning — **double descent** and **grokking** — on a Transformer small enough to train on your laptop, then explain them with the exact bias-variance and regularization machinery in your notes.

Theme. Training dynamics & efficiency.

CS229 anchors. - **Ch. 8 — Generalization** (§8.1 bias-variance tradeoff, §8.1.1 the formal decomposition for regression, **§8.2 the double descent phenomenon** — model-wise *and* sample-wise, Figs. 8.10–8.12, §8.3 sample complexity). Your notes discuss this at length and even show the *linear-model* double descent. - **Ch. 9 — Regularization** (§9.1 L2/weight decay, §9.2 implicit regularization). The notes explicitly note that regularization can *mitigate the double-descent peak* (Fig. 8.11) — you'll verify this. - **Ch. 7 — Deep learning** (§7.2–7.4): you need a working (small) neural net and its training loop; backprop understanding underpins the whole thing.

The bridge. §8.2 introduces double descent as an active research topic and even walks through it for **linear models with random features** (Fig. 8.12's setup). Grokking — where a small Transformer trained on modular arithmetic suddenly generalizes *long after* it has memorized the training set — is the LLM-era face of the same "more training/'capacity' keeps helping past the interpolation threshold" story. Because the task is tiny (e.g. `a + b mod p`), the whole thing fits in well under 8 GB and trains in minutes to hours, so you can map out entire curves that would be impossible with a real LLM.

The math you'll own. The bias-variance decomposition (§8.1.1) and how to *estimate each term by Monte Carlo* (train many models on resampled data); why the test error can peak at the interpolation threshold and fall again (§8.2); how weight decay (§9.1) and implicit regularization (§9.2) reshape the curve; the notion of an interpolation threshold (params ≈ data).

Build it from scratch. - **The linear/random-feature double descent** exactly as in the notes (§8.2, Fig. 8.12): random-feature ridge regression, sweep the number of features across the interpolation threshold, plot test error and the norm of the learned weights. This is your *theoretical anchor*, fully from scratch (NumPy). - **A bias-variance Monte-Carlo estimator** (§8.1.1): train an ensemble on resampled datasets; decompose test MSE into bias² + variance + noise; watch variance blow up at the threshold. - **A small Transformer** for grokking on modular arithmetic. Here you may use PyTorch `nn` modules as *plumbing*, but you write the **training loop, the weight-decay term, and all the logging** yourself, and you should be able to explain each module against Ch. 7. - *Plumbing:* PyTorch layers, optimizer, plotting.

Models & data. - Synthetic algorithmic data: modular addition/multiplication `(a, b)` → `(a∘b) mod p` for prime `p` (e.g. p=97), the canonical grokking dataset. No downloads,

no license issues, tiny. - Optional real-model tie-in: load two sizes of `pythia` and show train/test loss vs. scale to connect the toy curve to genuine LLMs (analysis only, no training).

Milestones. 1. *MVP:* random-feature ridge regression double-descent curve reproduced from the notes' own setup. 2. *Bias-variance:* Monte-Carlo decomposition showing the variance spike at the interpolation threshold. 3. *Grokking:* train the tiny Transformer on modular arithmetic; produce the iconic plot where **train accuracy hits 100% early but test accuracy jumps to ~100% much later**. 4. *Control knob:* sweep **weight decay** and show it controls the grokking delay / the double-descent peak (your notes' Fig. 8.11 claim, on a Transformer). 5. *Portfolio-grade:* a single narrative — "the same phenomenon, three ways: linear theory (from the notes) → bias-variance decomposition → a real (tiny) Transformer" — with reproducible scripts and seeds.

Theory vs. reality. §8.2 says regularization mitigates the peak and that the second descent is "not fully explained." You'll get to *see* the peak, *tame* it with weight decay, and stare at the still-mysterious second descent on a model you trained — exactly the frontier the notes point at.

The research question. *"Is grokking on modular arithmetic just sample-wise/model-wise double descent in disguise — i.e., can I place the memorize-then-generalize transition on the same axis as the interpolation-threshold peak, and does weight decay move both the same way?"*

Scope & pitfalls. Grokking is **seed- and hyperparameter-sensitive** — budget time for sweeps and fix seeds. Keep `p` small and the model tiny (a 1–2 layer, ~d=128 Transformer groks fine). Log continuously; the interesting behavior happens over many steps *after* apparent convergence, so don't stop training early.

Effort. ~3 weekends (plus some patient GPU-hours for the grokking sweeps — cheap on your card because the model is tiny).

Project 6 — RLHF from Scratch: steering a language model with REINFORCE

Hook. Implement the policy-gradient algorithm behind RLHF from first principles and use it to steer a small language model's generations toward a reward (positive sentiment, a target length, or a formatting rule) — the classical-RL chapters of your notes become the training method behind modern aligned LLMs.

Theme. Training dynamics / behavior (and the modern technique most worth having on your résumé for LLM research).

CS229 anchors. - **Ch. 17 — Policy Gradient (REINFORCE):** the core algorithm — the log-derivative trick, the score-function estimator, baselines for variance reduction. This *is* the "RL" in RLHF. - **Ch. 15 — Reinforcement learning / MDPs** (§15.1 MDP formalism, §15.2 value/policy iteration for grounding, §15.4 continuous/large state spaces → function approximation). You'll frame text generation as an MDP. - **Ch. 14**

— Autoregressive generation (Eqs. 14.9–14.16): the LLM's sampling process *is* the policy; temperature is your exploration knob.

The bridge. RLHF/RLAIF — the alignment step behind essentially every deployed chat model — is policy gradient (Chapter 17) applied to an LLM whose **policy** is the autoregressive next-token distribution `π_θ(x_t | x_{<t}) = softmax(f_θ(·))` from §14.3. Framing decoding as an MDP (state = prefix, action = next token, reward = a score on the completion) turns your notes' RL chapters into the training loop for a language model. You'll build a minimal, transparent RLHF — no black-box `trl` — so you understand every term.

The math you'll own. The policy-gradient theorem and the REINFORCE estimator `∇_θ J = E[∇_θ log π_θ(a|s) · (R − b)]` (Ch. 17); why a **baseline** `b` reduces variance without adding bias; the role of a **KL penalty** to the reference policy (why RLHF keeps the model from collapsing) and how it connects to regularization (Ch. 9); the MDP framing (Ch. 15).

Build it from scratch. - **The REINFORCE loop:** sample completions from the policy, score them, compute the log-prob of each sampled token (`llmkit/logits.py`), form the policy-gradient loss, backprop. Written by hand — this is the point. - **Variance reduction:** a moving-average baseline and/or a learned value baseline; measure the variance reduction empirically (a great plot). - **A KL-to-reference penalty** (the RLHF stabilizer); sweep its coefficient and show the reward-vs-KL tradeoff (the canonical RLHF curve). - **Reward functions** (start rule-based, no reward model needed): a sentiment classifier score, a target-length reward, or "contains/avoids token X." *Stretch:* train a tiny reward model from preference pairs (reuse logistic regression) for a true 3-stage RLHF. - *Plumbing:* HF model + LoRA adapter (so you fine-tune cheaply on 8 GB), generation utilities, optimizer.

Models & data. - Policy model: `gpt2` or `pythia-160m/410m` with a **LoRA** adapter — small enough that REINFORCE is stable and fast on 8 GB. - Reward: an off-the-shelf sentiment classifier (plumbing) or your own rule; preferences (stretch) can be synthetic.

Milestones. 1. *MVP:* REINFORCE steers `gpt2` toward a trivial reward (e.g. longer or more-positive completions); reward goes up over training. 2. *Variance reduction:* add a baseline; plot gradient variance and sample efficiency with vs. without it (your notes' baseline claim, verified). 3. *Stay on the manifold:* add the KL penalty; show that without it the model **reward-hacks** into gibberish, and with it, quality holds — plot the reward-vs-KL frontier. 4. *Portfolio-grade:* a clean writeup "RLHF in 300 lines," with the reward curves, the KL tradeoff, and before/after generation samples. *Stretch:* full 3-stage pipeline with a learned reward model from preference pairs.

Theory vs. reality. Ch. 17 promises the score-function estimator is unbiased but **high-variance**; you'll *feel* that variance (training is noisy) and *fix* it with baselines exactly as the theory says. You'll also discover the thing the notes don't emphasize — that reward maximization alone destroys the model (reward hacking), which is *why* real RLHF needs the KL term.

The research question. *"How does the reward-vs-KL Pareto frontier change with model size and with the exploration temperature — i.e., how much 'alignment' can*

you buy before the policy drifts too far from the base model?"

Scope & pitfalls. REINFORCE is genuinely noisy — small learning rate, LoRA (not full fine-tune), and a baseline are near-mandatory. Keep sequences short (reward on ≤50 tokens). Watch for reward hacking as a *feature to study*, not just a bug. Don't reach for PPO first; get plain REINFORCE + baseline + KL working, then mention PPO as the industrial upgrade.

Effort. ~3–4 weekends. The highest "sounds impressive and actually is" ratio of the set.

Appendix A — Suggested order & dependencies

You don't have to do all six, but they're designed to compound. Recommended path:

1. **Shared toolkit (`llmkit`)** — half a day; unblocks everything.
2. **Project 1 (Probes)** — builds `probe.py` (logistic/softmax regression) that Projects 3 and 4 reuse. Best first real project.
3. **Project 2 (Geometry)** — builds `linalg.py` (PCA/ICA); independent of 1 but shares the activation harness.
4. **Project 3 (Calibration)** — reuses `probe.py`; short and high-insight; good momentum.
5. **Project 4 (Detection)** — reuses `probe.py` and log-prob utilities; the SMO implementation is the big lift.
6. **Project 5 (Double descent)** — mostly standalone; needs patience more than code.
7. **Project 6 (RLHF)** — most involved; do it once you're comfortable with generation + LoRA.

Dependency sketch:

```
llmkit  ──►  P1 (probe.py) ──►  P3
   │              └──────────►  P4  ◄── (SMO, GDA, NB: new)
   ├──►  P2 (linalg.py)
   ├──►  P5  (independent)
   └──►  P6  (independent, needs LoRA + generation)
```

A realistic pace for one developer treating this as a serious side-project: **one project every 2–4 weekends**, so the full set is a ~4–6 month arc. Doing **any three** (e.g. 1, 3, 6 — one per theme) already makes a coherent portfolio story: *"I can probe what a model knows, measure whether it's honest, and align it."*

Appendix B — Environment setup for 8 GB VRAM

A single environment covers all six projects.

```
# CUDA-enabled PyTorch (match your CUDA version)
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

```
pip install transformers datasets accelerate
pip install bitsandbytes           # 4-bit/8-bit quantization for the
 ↪  bigger models
pip install peft                   # LoRA/QLoRA for Project 6 (and any
 ↪  fine-tuning)
pip install numpy scipy matplotlib scikit-learn   # scipy/sklearn =
 ↪  plumbing & sanity-checks only
pip install sentence-transformers   # Project 2 semantic-similarity eval
```

Practical 8 GB habits:

- Load big models 4-bit: `AutoModelForCausalLM.from_pretrained(name, load_in_4bit=True, device_map="auto")`. A 7B model lands around ~4–5 GB this way (inference only).
- Prefer **fp16/bf16** for ≤1.4B models; they fit natively.
- For activation extraction, run **`torch.inference_mode()`**, process in **small batches**, and immediately move captured tensors to CPU — hidden states for a big batch are what actually blow the budget, not the weights.
- For any training (Projects 5–6), use **gradient checkpointing**, **LoRA** (never full fine-tune a big model here), batch size 1–4 with gradient accumulation, and short sequence lengths.
- Use **`sklearn` only as a checker** — e.g. confirm your from-scratch PCA matches `sklearn`'s to numerical tolerance. The submission uses *your* implementation.
- 32 GB system RAM comfortably holds cached activations/embeddings for the classical-ML steps, which run on **CPU** — offload the linear algebra there and keep the GPU for the forward passes.

Appendix C — Why Chapter 16 (LQR/DDP/LQG) is left out

Chapters 15 and 17 (MDPs and policy gradient) bridge cleanly to LLMs through RLHF, so they're in (Project 6). **Chapter 16 — LQR, DDP, LQG** — is about optimal control of continuous dynamical systems with quadratic cost and Gaussian noise. It's beautiful and central to robotics/control, but it does not map naturally onto language-model research, and forcing a bridge would produce exactly the kind of contrived project you asked to avoid. Leaving it out is the honest call. (If you ever pivot toward control or RL-for-robotics, Chapter 16 becomes a first-class project on its own terms.)

Appendix D — One real paper per project (to anchor the "research" framing)

Each project deliberately shadows a real line of work, so you can read the source, compare, and cite it in a writeup. Titles given so you can search them:

- **P1 (Probes):** Alain & Bengio, *"Understanding intermediate layers using linear classifier probes."* Hewitt & Liang, *"Designing and Interpreting Probes with*

Control Tasks" (the selectivity idea). "Logit lens" (nostalgebraist, blog).

- **P2 (Geometry):** Mu & Viswanath, *"All-but-the-Top: Simple and Effective Post-processing for Word Representations."* Ethayarajh, *"How Contextual are Contextualized Word Representations?"* (anisotropy). Recent ICA-on-embeddings work on "universal geometry."
- **P3 (Calibration):** Guo et al., *"On Calibration of Modern Neural Networks"* (temperature scaling). Desai & Durrett / Kadavath et al. on LLM calibration and "language models (mostly) know what they know."
- **P4 (Detection):** Mitchell et al., *"DetectGPT"* (log-prob curvature). Your notes' §4.1.3 (Ng) for the generative-vs-discriminative theory.
- **P5 (Double descent / grokking):** Nakkiran et al., *"Deep Double Descent."* Power et al., *"Grokking."* Belkin et al. (referenced in your notes' §8.2 bibliography).
- **P6 (RLHF):** Williams, *"Simple statistical gradient-following algorithms"* (REINFORCE). Ziegler et al. / Stiennon et al. / Ouyang et al. (InstructGPT) for the RLHF pipeline.

Appendix E — What "portfolio-grade" means for these

For each project, the difference between a toy and a portfolio piece is the same three things: **(1) a control or baseline** that rules out the boring explanation (the control task in P1, the data-size sweep in P4, the seed sweep in P5); **(2) a figure that tells the story in one glance** (layer-vs-accuracy, reliability diagram, AUC-vs-data-size, the grokking plot, the reward-vs-KL frontier); and **(3) a short, honest writeup** — 1–2 pages or a blog post — that states what the theory predicted, what you observed, and where they diverged. Ship each project as its own small repo with a README, fixed seeds, and a `make reproduce` (or one script) that regenerates the headline figure. That reproducibility is what signals "research engineer," not "tutorial follower."

Appendix F — Datasets & models (copy-paste ready)

Everything you need to actually start each project: exact Hugging Face IDs, a load snippet, how much to use (so you stay within an 8 GB / laptop budget), what to extract, the license situation, and an **offline / build-your-own fallback** in case a Hub ID or loader has drifted.

Two caveats up front (my knowledge is current only to mid-2025, and the Hub moves): - If a `load_dataset(...)` call complains about a script, add `trust_remote_code=True`, or grab the script-free mirror the fallback lists. If an **ID 404s**, search the Hub — canonical datasets sometimes move under an org (e.g. `sst2` → `stanfordnlp/sst2`). - Licenses below are the common case; **check each dataset/model card before redistributing** anything. For a personal portfolio repo, ship your *code and figures*, not the raw data.

A one-time install for the data side (the rest is in Appendix B):

```
pip install datasets huggingface_hub
```

P1 — Linear Probes

Property	Dataset	HF ID + load	Use	License
Sentiment (binary)	SST-2 (GLUE)	`load_dataset("glue","sst2")` → `sentence,label`	~67k train, 872 val as test	permissive (research)
Part-of-speech (multi-class)	Universal Dependencies EWT	`load_dataset("universal_dependencies","en_ewt",trust_remote_code=True)` → `tokens,upos`	~12k sentences	CC BY-SA
Factual truth (binary)	*geometry-of-truth* CSVs	Samuel Marks's `geometry-of-truth` GitHub repo, files like `cities.csv` `(statement,label)`	1–2k statements	MIT (code)
Syntactic agreement (paired)	BLiMP	`load_dataset("blimp","irregular_plural_subject_verb_agreement_1")`	1k minimal pairs	CC BY

```
from datasets import load_dataset
sst2 = load_dataset("glue", "sst2")           # sst2["train"],
 ↪  sst2["validation"]
# extract: run model on `sentence`, grab hidden state at the FINAL token,
 ↪  every layer
```

What to extract: for each example, `get_activations(model, [text], layers=all)` and keep the vector at the last (or target-word) token position. That matrix (examples × d) is your probe input. **Fallbacks:** POS → `load_dataset("eriktks/conll2003")` uses `pos_tags` (Penn tags), script-free. Truth → generate 500 statements from a template (`"The capital of {country} is {city}."`, flip half to false); this is fully controllable and arguably cleaner.

P2 — The Shape of Meaning

Purpose	Source	Load	Notes
Static token embeddings	the model itself	`model.get_input_embeddings().weight.detach()`	`[vocab, hidden]` matrix

Purpose	Source	Load	Notes
Semantic-similarity eval	STS-B (GLUE)	`load_dataset("glue","stsb")` → `sentence1,sentence2,label` (0–5)	cosine vs. gold (Spearman)
Contexts for polysemy / anisotropy	WikiText-103	`load_dataset("Salesforce/wikitext","wikitext-103-raw-v1")`	sample ~1k thousand sentences

```
sts = load_dataset("glue", "stsb")            # measure
 ↪  Spearman(cosine(emb1,emb2), label)
wiki = load_dataset("Salesforce/wikitext", "wikitext-103-raw-v1",
 ↪  split="train[:2%]")
```

What to extract: for "all-but-the-top", embed each STS sentence (mean-pool a layer, or use `all-MiniLM-L6-v2`), remove top-k PCA components, re-score. For senses, collect ~30 contextual vectors each for polysemous words ("bank", "spring", "cell") pulled from WikiText, then cluster. **Fallback:** if WikiText's config errors, `load_dataset("wikimedia/wikipedia","20231101.en",split="train[:0.1%]")`, or hand-write 20 sentences per sense.

P3 — Calibration

Benchmark	HF ID	Fields	Use
MMLU	`load_dataset("cais/mmlu","all")`	`question,choices,answer`	1–2k Qs; answer = A/B/C/D single token
ARC-Challenge	`load_dataset("allenai/ai2_arc","ARC-Challenge")`	`question,choices,answerKey`	hard good contrast
HellaSwag	`load_dataset("Rowan/hellaswag")`	`ctx,endings,label`	commonsense

```
mmlu = load_dataset("cais/mmlu", "all")       # use validation to FIT
 ↪  temperature, test to REPORT ECE
# read logits of the " A"/" B"/" C"/" D" tokens; softmax over the 4 →
 ↪  predicted prob
```

Model pair (the key comparison): `Qwen/Qwen2.5-1.5B` vs `Qwen/Qwen2.5-1.5B-Instruct` (base vs instruction-tuned, same size). Both run fp16 on 8 GB. **License:** MMLU/ARC/HellaSwag are research-permissive; Qwen2.5 is Apache-2.0. **Fallback:** if you want everything offline, ARC-Easy is small and downloads fast.

P4 — Who Wrote This?

Role	Source	Load
Ready-made human vs. machine	HC3 (Human ChatGPT Comparison Corpus)	`load_dataset("Hello-SimpleAI/HC3","all")` → `human_answers,chatgpt_answers`
Human reference corpus (self-gen setup)	WikiText-103	`load_dataset("Salesforce/wikitext","103-raw-v1")`
Machine text	**you generate it**	sample from a small model at temps {0.7, 1.0, 1.3}

```
hc3 = load_dataset("Hello-SimpleAI/HC3", "all")   # instant labeled pairs
 ↪  to start
# self-generated set: take a human passage prefix, let gpt2/pythia continue
 ↪  ~200 tokens
```

Feature extraction (the LLM part): for each text, use `llmkit.logits.token_logprobs` to compute mean log-prob, log-prob variance, and a DetectGPT-style **curvature** feature (drop in mean log-prob after a few masked-token resamples). Those features feed your from-scratch NB / GDA / logistic / SMO-SVM. **Critical:** match human and machine texts on **topic and length**, or you'll classify the topic. HC3 is pre-paired (both answer the same question), which controls topic for you — start there. **License:** HC3 research-use; WikiText CC BY-SA.

P5 — Double Descent & Grokking **No download.** The data is synthetic and generated in a few lines — this is why it fits your GPU:

```
import itertools, torch
p = 97                                              # prime modulus
pairs = list(itertools.product(range(p), range(p)))
X = torch.tensor(pairs)                             # (p*p, 2) tokens a,b
y = torch.tensor([(a + b) % p for a, b in pairs]) # target (a+b) mod p
# shuffle, take a fraction (e.g. 30–50%) as train to induce grokking; rest
 ↪  is test
```

For the **linear/random-feature double descent** anchor, generate synthetic regression data (or reuse SST-2 features from P1) and sweep the number of random features across the interpolation threshold. Nothing to download.

P6 — RLHF from Scratch

Role	ID	Notes
Policy model	`gpt2` or `EleutherAI/pythia-410m`	+ a LoRA adapter (`peft`) so training fits 8 GB
Reward: sentiment	`lvwerra/distilbert-imdb` (or `distilbert-base-uncased-finetuned-sst-2-english`)	frozen; scores completions
Prompts	`load_dataset("stanfordnlp/imdb")`	use the first ~8 tokens of a review as the prompt
Preference pairs (*stretch* — train your own reward model)	`Dahoas/rm-static` or `Anthropic/hh-rlhf`	large — sample a few thousand pairs only

```
imdb = load_dataset("stanfordnlp/imdb", split="train")   # prompt =
 ↪  review[:few tokens]
# reward = sentiment_model(completion) → scalar; REINFORCE loss =
 ↪  -logπ_θ(sampled)·(R-baseline) + βKL
```

Rule-based rewards need no model at all (target length, "contains word X", regex format) — start there to debug the REINFORCE loop, then swap in the sentiment reward. **License:** GPT-2/Pythia and the DistilBERT rewards are permissive; `hh-rlhf` is MIT (Anthropic).

Consolidated model IDs (all 8 GB-safe)

```
Probing / small analyzable:    gpt2, gpt2-medium, EleutherAI/pythia-
{70m,160m,410m,1.4b}
Base vs instruct pair:      Qwen/Qwen2.5-1.5B & Qwen/Qwen2.5-1.5B-Instruct
4-bit "big" comparison:          meta-llama/Llama-3.2-3B, microsoft/phi-
2  (load_in_4bit=True)
Sentence embeddings:         sentence-transformers/all-MiniLM-L6-v2
Reward (P6):                 lvwerra/distilbert-imdb
```

If you'd like, I can run a quick live check to confirm each ID resolves on today's Hub before you start — the canonical ones above are stable, but it's a 30-second sanity pass.